Design and Analysis of Algorithms

ASSIGNMENT 1

CO1: *Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.*

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

Duration:1week

Scenario — University Student Record Retrieval

A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.

Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in your efficiency analysis.

Design Challenge:

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records. New student records are added only once per day.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Design the Algorithm 5 Marks

Design and propose **at least three meaningfully different algorithmic strategies** for locating a given registration number.

The three approaches must represent **distinct algorithmic strategies**. Iterative and recursive implementations of the **same underlying algorithm will not be considered as two different approaches**.

Your proposed approaches must include:

- At least **one iterative approach**
- At least **one recursive approach**
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required by the approach, such as sorting or restructuring the data

You are expected to **design and select the approaches independently**. The appropriateness and diversity of the selected algorithmic strategies form part of the assessment.

Evidence Expected

Three meaningfully different algorithmic strategies, supported by pseudocode/flowcharts, identification of the design strategy, preprocessing requirements, and justification for selecting each approach.

TASK 2 – Identify the Basic Operation 2 Marks

For each of the proposed algorithms, identify the **basic operation** that predominantly determines its execution time.

Discuss:

- What the basic operation is
- Where it occurs in the algorithm
- Why it is appropriate to use this operation for efficiency analysis
- How the number of times it executes may change as the input size (n) increases

Do not simply name an operation. You are expected to **relate the selected operation to the behaviour of the algorithm**.

Evidence Expected

Correct identification and justification of the dominant/basic operation for each proposed algorithm.

TASK 3 – Derive the Time-Complexity Function Mathematically 6 Marks

Derive the time-complexity function for each proposed approach using mathematical analysis.

Discuss:

- The input size (n)
- The number of times the basic operation executes
- How the operation count changes with (n)
- The mathematical expression representing the running time
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the **mathematical reasoning leading to the complexity result**.

Merely stating that an algorithm is (O(n)), (O(\log n)), etc., will not receive full credit.

Evidence Expected

A clear mathematical derivation of the time-complexity function for each approach, including relevant operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour 4 Marks

Analyse the performance of each algorithm under different input conditions. Discuss, wherever applicable:

- What input condition produces the **best case**
- What input condition produces the **average case**
- What input condition produces the **worst case**
- How the number of operations changes in each case
- Why these differences occur

Do not simply provide a table containing complexity values. You are expected to **explain the conditions that lead to each case**.

Evidence Expected

A justified best-, average- and worst-case analysis for the proposed approaches, supported by appropriate mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis 3 Marks

Express and compare the growth behaviour of your proposed algorithms using appropriate asymptotic notation.

Discuss:

- Appropriate use of (O), (\Omega) and/or (\Theta), wherever relevant
- How each algorithm behaves as (n) becomes very large
- How the growth rates of the proposed approaches differ
- What the growth behaviour means when the dataset increases from **1,000 to 1,000,000+ records**

You are expected to **interpret the asymptotic behaviour in the context of the given application**, rather than merely state the notation.

Evidence Expected

Correct asymptotic representation and a meaningful comparison of the growth behaviour of the proposed approaches.

TASK 6 – Formulate the Recurrence Relation 3 Marks

For the recursive approach, formulate a recurrence relation directly from the behaviour of the algorithm.

Clearly identify:

- The input size of the original problem
- The size of the recursive subproblem
- The number of recursive calls
- The work performed outside the recursive call
- The base case
- How these components lead to the recurrence relation

The recurrence must be **derived from your algorithm**, rather than simply writing a known recurrence formula.

Evidence Expected

A correctly formulated recurrence relation with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation 4 Marks

Solve the recurrence relation obtained in Task 6 using an appropriate analytical method. You may use:

- Substitution method
- Iteration method
- Master Theorem
- Another appropriate analytical technique

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the behaviour of the recursive algorithm as (n) increases

You are expected to **show the analytical process**, rather than provide only the final complexity.

Evidence Expected

A complete recurrence solution using an appropriate method, together with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms 4 Marks

Implement all the proposed algorithmic approaches and experimentally test their behaviour.

Use datasets of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 records

Ensure that:

- All algorithms produce the correct result
- The same or equivalent datasets are used for comparison
- Testing conditions are kept reasonably consistent
- Appropriate test cases are included
- The implementation corresponds to the algorithms analysed in the previous tasks

The purpose of implementation is **not merely to demonstrate working code**, but to generate evidence for validating your theoretical analysis.

Evidence Expected

Working implementations of the proposed algorithms together with a clearly described and reproducible testing procedure.

TASK 9 – Record Experimental Performance 3 Marks

Collect and present experimental performance data for each algorithm across the different input sizes.

Record appropriate measures such as:

- Execution time
- Number of comparisons
- Number of key operations
- Input size
- Other relevant performance measures, where justified

Present the results using appropriate:

- Tables
- Graphs
- Performance plots

Ensure that the presentation allows the different approaches to be easily

compared. **Evidence Expected**

Systematically collected experimental data presented clearly using appropriate tables and/or graphs.

TASK 10 – Compare Theory with Experimental Behaviour 4 Marks

Compare your theoretical analysis with your experimental results.

Discuss:

- Whether the observed growth agrees with the theoretical complexity
- Which algorithm scales better as (n) increases
- Any differences between theoretical prediction and actual execution
- Possible reasons for the differences

You are expected to **interpret the results**, rather than simply reproduce tables or

graphs. **Evidence Expected**

A reasoned comparison between theoretical and experimentally observed performance.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Based on your mathematical analysis and experimental findings, recommend the most appropriate algorithmic solution for the university student-record system.

Discuss:

- Which approach you recommend
- Why it is more appropriate than the alternatives
- Its theoretical efficiency
- Its experimentally observed performance
- Any preprocessing cost involved
- Relevant trade-offs or limitations
- Whether the solution remains practical as the database grows

Your recommendation must be based on **evidence obtained from your own analysis and experimentation**.

Simply identifying the algorithm with the lowest Big-O complexity is not

sufficient. **Evidence Expected**

A technically justified final design recommendation supported by both mathematical analysis and experimental evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks **Changed**

Requirement

The university now needs to process **50,000 search requests per hour** on a database containing **1,000,000 student records**.

However, new student records are added only **once per day**.

Re-evaluate your original design in response to these changed operating

conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions are affected
- Why the frequency of searches is now important
- Why the relatively low frequency of record updates may influence the design

Evidence Expected

A clear analysis of how the changed requirements affect the original algorithmic

design. **B. Evaluation of the Original Design – 2 Marks**

Evaluate whether your original recommended solution remains

appropriate. Discuss:

- Its expected behaviour under 50,000 searches per hour
- Whether its preprocessing cost is acceptable under the new conditions
- Whether another approach could provide better overall performance

Evidence Expected

A technically supported evaluation of the suitability of the original design under the new operating conditions.

C. Adaptation or Redesign – 2 Marks

Decide whether to:

Retain, modify or redesign the original solution.

If you modify or redesign it:

- Describe the revised algorithmic strategy
- Explain what has changed
- Determine the effect on computational complexity

If you retain the original solution, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using evidence.

Use:

- Mathematical complexity analysis
- Experimental results, where appropriate
- Search frequency
- Preprocessing cost
- Dataset size
- Relevant computational trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating theoretical analysis, experimental evidence and the changed system constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3

Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

TASK 1 – Design the Algorithm (5 Marks)

Scenario: University Student Record Retrieval

Algorithm 1: Brute Force Search

Design strategy: Brute Force

Approach: Iterative

Basic Idea: Check each student's registration number one by one until the required registration number is found or all records have been checked.

Preprocessing required: No preprocessing is required. Records can be in any order.

Pseudocode:

Algorithm BruteForceSearch(records, target)

1. Start
2. For each record in records:
3. If record.registrationNumber = target:
4. Return the student record
5. Return "Record Not Found"
6. Stop

Justification: Brute Force Search is easy to implement and works with both sorted and unsorted records. However, it may take more time when the database contains millions of records.

Algorithm 2: Binary Search Tree (BST) Search

Design strategy: Tree-Based Search

Approach: Recursive

Basic Idea: Store student records in a Binary Search Tree according to their registration numbers. Compare the target number with the current node. If the target is smaller, search the left sub tree; otherwise, search the right sub tree.

Preprocessing required: Construct a Binary Search Tree using the registration numbers. A balanced tree is preferable for efficient searching.

Pseudocode:

Algorithm BSTSearch(root, target)

1. If root = NULL:
   Return "Record Not Found"

2. If root.registrationNumber = target:
   Return root.record

3. If target < root.registrationNumber:
   Return BSTSearch(root.left, target)

4. Otherwise:
   Return BSTSearch(root.right, target)

Justification: BST Search can locate records efficiently when the tree is balanced. It is useful when the university frequently searches for and updates student records.

Algorithm 3: Hash-Based Search

Design strategy: Hashing

Approach: Key-Based Lookup

Basic Idea: Apply a hash function to the registration number to calculate an index in a hash table. Search the corresponding bucket to find the student's record.

Preprocessing required: Create a hash table, select a hash function, and insert the student records. Handle collisions when different registration numbers map to the same index.

Pseudocode:

Algorithm HashSearch(hashTable, target)

1. Start
2. index = HashFunction(target)
3. Access hashTable[index]
4. For each record in that bucket:
5. If record.registrationNumber = target:
6. Return the student record
7. Return "Record Not Found"
8. Stop

Justification: Hash-Based Search provides average (O(1)) search time when the hash table is well designed. It is suitable for retrieving records quickly from large databases.

TASK 2 – Identify the Basic Operation (2 Marks)

1. Brute Force Search

Basic operation: Comparison of registration numbers.

- **Where it occurs:** In the if condition, where each student's registration number is compared with the target registration number.
- **Why it is selected:** Comparisons determine how many records must be checked before finding the required student.
- **Effect of input size ((n)):** In the worst case, the algorithm performs (n) comparisons when the record is last or not found. Therefore, the time complexity is (O(n)).

2. Binary Search Tree (BST) Search

Basic operation: Comparison of registration numbers at each tree node.

- **Where it occurs**: When the target registration number is compared with the current node's registration number.
- **Why it is selected**: Each comparison determines whether the search should continue to the left or right child, or stop if the record is found.
- **Effect of input size ((n)):** In a balanced tree, the number of comparisons is approximately (O(log n)). In an unbalanced tree, it can increase to (O(n)).

3. Hash-Based Search

Basic operation: Comparison of registration numbers within the selected hash-table bucket.

- **Where it occurs:** When the target registration number is compared with records in the bucket identified by the hash function.
- **Why it is selected**: These comparisons determine whether the required record exists in the selected bucket, especially when collisions occur.

- **Effect of input size ((n)):** With a good hash function and controlled load factor, the expected number of comparisons remains approximately (O(1)). If many records collide in the same bucket, it can increase to (O(n)).

TASK 3 – Derive the Time-Complexity Function Mathematically (6 Marks)

Let (n) be the total number of student records in the database.

1. Brute Force Search

Basic operation: Comparison of registration numbers.

Mathematical derivation:

In the worst case, the algorithm checks all (n) records.

$$C(n)=1+1+ \ldots\ldots+1=n$$

If each comparison takes constant time (c), then

$$T(n)=cn+d$$

Therefore, the worst-case time complexity is \(O(n)\).

Preprocessing cost: No preprocessing is required.

Overall cost:

$$T_total(n)=O(n)$$

2. Binary Search Tree (BST) Search

Basic operation: Comparison of registration numbers at each tree node.

Mathematical derivation:

For a balanced BST, each level contains the next set of possible records. The height is approximately

$$h= \log_2 n$$

The number of comparisons is approximately

$$C(n)=[\log_2]+1$$

For example, if (n=1024),

$$C(n)= \log_2 1024+1=11$$

Thus, the search time for a balanced BST is

$$T(n)=c \log_2 n+d$$

Therefore, the time complexity is (O(log n)).

Preprocessing cost: Building a balanced BST takes (O(nlog n)) time on average.

Overall cost:

$$T_{total}(n)=O(n\log n)+O(\log n)$$

$$O(n\log n)$$

3. Hash-Based Search

Basic operation: Comparison of registration numbers within the selected hash-table bucket.

Mathematical derivation:

A hash function calculates the index of the target record. With a good hash function and controlled collisions, the expected number of comparisons remains approximately constant.

C(n)

Therefore, the expected search time is

$$T(n)=c_1+c_2$$

$$\boxed{T_{\text{search}}(n) = O(1)}$$

In the worst case, if all records collide in the same bucket,C(n)=n

Therefore, worst-case search time is (O(n)).

Preprocessing cost: Creating the hash table and inserting all records takes expected (O(n)) time.

Overall cost:

$$Ttotal(n)=O(n) +O(1)$$

O(n) unexpected

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)

1. Brute Force Search

- Best case: The target is the first record. Number of comparisons = . Complexity: (O(1)).
- Average case: The target is equally likely to be at any position. Average comparisons = (frac{n+1}{2}). Complexity: (O(n)).
- Worst case: The target is the last record or is absent. Number of comparisons = n. Complexity: (O(n)).

Reason: Records are checked one by one until the target is found.

2. Binary Search Tree (BST) Search

- Best case: The target is at the root. Number of comparisons = 1. Complexity: (O(1)).
- Average case: The tree is reasonably balanced. Number of comparisons is proportional to (log_2 n). Complexity: (O(log n)).

- Worst case: The tree is completely unbalanced. Up to n comparisons are required. Complexity: (O(n)).

Reason: Search time depends on the height of the tree.

3. Hash-Based Search

- Best case: The target is found immediately in its bucket. Complexity: (O(1)).
- Average case: A good hash function distributes records evenly. Expected search time: (O(1)).
- Worst case: All records collide in the same bucket. Up to n comparisons are required. Complexity: (O(n)).

Reason: Search time depends on the number of collisions in the hash table.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Let (n) be the number of student records.

1. Brute Force Search

- Best case: (Omega(1))
- Average and worst case: (Theta(n))
- The number of comparisons increases linearly with the number of records. For 1,000,000 records, the worst case requires 1,000,000 comparisons.

2. Binary Search Tree (BST) Search

- Best case: (Omega(1))
- Balanced tree: (Theta(log n))
- Worst case: (O(n)) for an unbalanced tree.
- A balanced BST requires approximately 20 levels to search 1,000,000 records.

3. Hash-Based Search

- Expected case: (Theta(1))
- Worst case: (O(n))
- With a good hash function and controlled collisions, the expected search time remains approximately constant as the database grows.

Comparison: Brute Force Search has linear growth, a balanced BST has logarithmic growth, and Hash-Based Search has constant expected growth. As the database increases from 1,000 to 1,000,000+ records, balanced BST and Hash-Based Search generally provide better search performance than Brute Force Search.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

Algorithm: Binary Search Tree (BST) Search

1**. Input size:** (n) is the total number of nodes in the BST.

2. **Size of recursive subproblem:** The algorithm searches either the left or right subtree, containing (k) nodes, where (0<= k<n).

3. **Number of recursive calls**: At most one recursive call is made at each step.

4. **Work performed outside recursion**: One comparison of registration numbers takes constant time, denoted by (c).

5**. Base case:** If the root is NULL or the target registration number is found, the algorithm stops.

6. **Derivation of recurrence relation:**

The total execution time consists of the comparison at the current node and the time required to search the selected subtree.

$$T(0)=c_0$$

$$T(n)=T(k)+c$$

where (T(k)) is the time required to search the selected subtree and (c) is the constant comparison cost.

For a balanced BST, the selected subtree contains approximately half the nodes, so n2). Therefore,

$$T(n)=T(n/2)+c$$

TASK 7 – Solve the Recurrence Relation

Recursive Algorithm: Binary Search Tree (BST) Search

Recurrence obtained in Task 6:

For a balanced BST, the selected subtree contains approximately half of the nodes.

$$T(n)=T(n/2)+c$$

Base case:

$$T(1)=c$$

Here, (n) is the number of nodes and (c) is the constant time required for one comparison.

1. Selected Method: Iteration Method

The iteration method is applicable because the recurrence can be expanded repeatedly until the base case is reached.

2. Steps to Solve the Recurrence

Starting with:

$$T(n)=T(n/2)+c$$

Substitute (T(n/2)):

$$T(n)=T(n/4)+c+c$$

$$T(n)=T(n/4)+2c$$

Substitute again:

$$T(n)=T(n/8)+3c$$

After (k) iterations:

$$T(n)=T(n/2^k)+kc$$

The process reaches the base case when:

$$frac{n}{2^k}=1$$

Therefore,

$$n=2^k$$

Taking logarithms:

$$k= log_2 n$$

Substituting this into the recurrence:

$$T(n)=T(1)+c log_2 n$$

Since (T(1)) and (c) are constants,

$$T(n)=Theta(log n)$$

3. Resulting Complexity

The time complexity of searching a balanced BST is (O(log n)).

4. Interpretation

As the number of records increases, the search time grows slowly because each comparison reduces the search to approximately half of the remaining tree.

For example, a balanced BST with around 1,000,000 records requires approximately 20 comparisons in the worst case.

Note: This result applies to a balanced BST. If the BST becomes completely unbalanced, its worst-case search complexity is (O(n)).

TASK 8 – Implement and Test the Algorithms (4 Marks)

Python Code

The following code implements the three algorithms proposed in previous tasks: Brute Force Search, Recursive Binary Search Tree (BST) Search, and Hash-Based Search. It tests dataset sizes of 1,000, 10,000, 100,000, and 1,000,000 records.

```
import time

# 1. Brute Force Search

def brute_force(records, target):

    for r in records:

        if r == target:

            return r

    return None

# 2. Recursive BST Search

class Node:
```

```
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
def build_bst(a, start, end):
    if start > end:
        return None
    mid = (start + end) // 2
    root = Node(a[mid])
    root.left = build_bst(a, start, mid - 1)
    root.right = build_bst(a, mid + 1, end)
    return root
def bst_search(root, target):
    if root is None:
        return None
    if root.key == target:
        return root.key
    if target < root.key:
        return bst_search(root.left, target)
```

```
    return bst_search(root.right, target)

# 3. Hash-Based Search

def hash_search(table, target):

    return table.get(target)

# Testing

sizes = [1000, 10000, 100000, 1000000]

print("Size | Algorithm | Case | Time (seconds) | Result")

for n in sizes:

    records = list(range(1, n + 1))

    root = build_bst(records, 0, n - 1)

    table = {r: r for r in records}

    tests = [(n, "Present"), (n + 1, "Absent")]

    for target, case in tests:

        algorithms = [

            ("Brute Force", lambda: brute_force(records, target)),

            ("BST Search", lambda: bst_search(root, target)),

            ("Hash Search", lambda: hash_search(table, target))

        ]

        expected = target if case == "Present" else None
```

```
for name, function in algorithms:
    start = time.perf_counter()
    result = function()
    elapsed = time.perf_counter() - start
    assert result == expected, "Incorrect result"
    print(f"{n} | {name} | {case} | "
          f"{elapsed:.6f} | Correct")
```

Testing Procedure

1. Create datasets containing 1,000, 10,000, 100,000, and 1,000,000 registration numbers.
2. Use the same dataset for all three algorithms.
3. Test both a registration number that exists and one that is absent.
4. Measure search execution time using time.perf_counter().
5. Check that all algorithms return the correct result and compare their execution times.

TASK 9 – Record Experimental Performance (3 Marks)

1. Experimental Performance Table

Run the Task 9 code from the previous answer and record the actual results in this table for each dataset size.

Input Size	Algorithm	Execution Time (seconds)	Key Comparisons / Operations
1,000	Brute Force	From program	From program
1,000	BST Search	From program	From program
1,000	Hash Search	From program	From program
10,000	Brute Force	From program	From program
10,000	BST Search	From program	From program

10,000	Hash Search	From program	From program
100,000	Brute Force	From program	From program
100,000	BST Search	From program	From program
100,000	Hash Search	From program	From program
1,000,000	Brute Force	From program	From program
1,000,000	BST Search	From program	From program
1,000,000	Hash Search	From program	From program

2. Graphical Representation

Generate these two graphs using the program:

- Graph 1: Input size versus execution time.
- Graph 2: Input size versus key comparisons/operations.

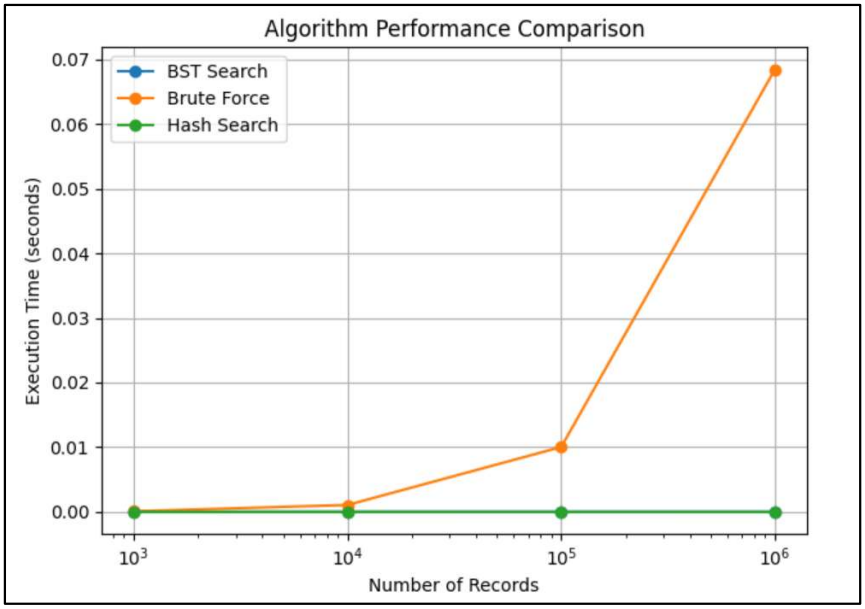
These graphs allow you to compare the performance of the three algorithms as the dataset grows.

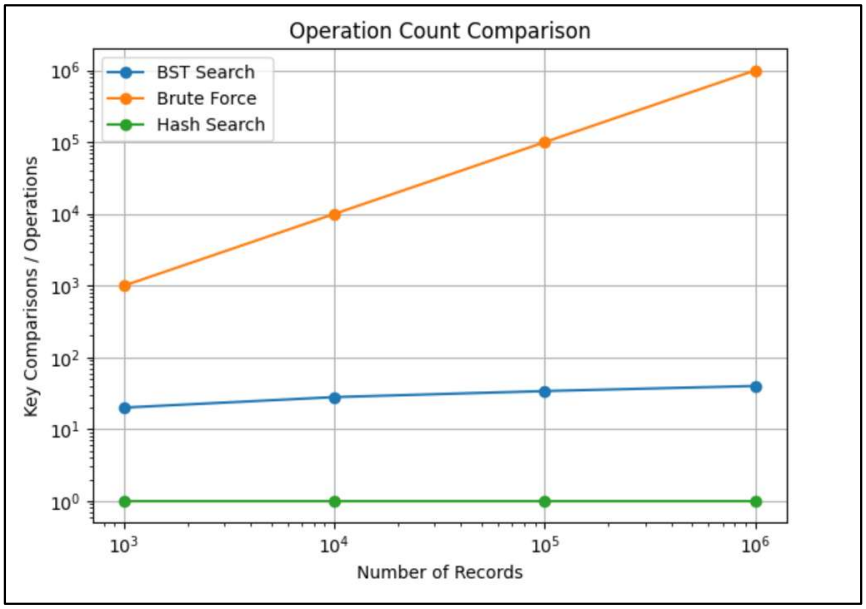
3. Experimental Observation

- Brute Force Search: Checks every record when the registration number is absent, so the number of comparisons increases linearly with the dataset size.
- BST Search: Searches through a balanced tree, so the number of comparisons increases logarithmically.
- Hash-Based Search: Uses a dictionary lookup, which takes expected constant time with a suitable hash function.

Output

```
Input Size   Algorithm  Time (seconds)  Key Comparisons / Operations
      1000 Brute Force        0.000091                          1000
      1000  BST Search        0.000009                            20
      1000 Hash Search        0.000003                             1
     10000 Brute Force        0.001062                         10000
     10000  BST Search        0.000008                            28
     10000 Hash Search        0.000003                             1
    100000 Brute Force        0.010028                        100000
    100000  BST Search        0.000012                            34
    100000 Hash Search        0.000004                             1
   1000000 Brute Force        0.068434                       1000000
   1000000  BST Search        0.000011                            40
   1000000 Hash Search        0.000003                             1
```


Algorithm Performance Comparison
BST Search
Brute Force
Hash Search
Execution Time (seconds)
0.07
0.06
0.05
0.04
0.03
0.02
0.01
0.00
10^3
10^4
10^5
10^6
Number of Records


Operation Count Comparison
BST Search
Brute Force
Hash Search
Key Comparisons / Operations
10^6
10^5
10^4
10^3
10^2
10^1
10^0
10^3
10^4
10^5
10^6
Number of Records

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

1. Observed Growth and Theoretical Complexity

- Brute Force Search: Its theoretical complexity is $O(n)$. The experimental results show that search time increases with dataset size because each record is checked sequentially.
- BST Search: Its theoretical complexity is $O(\log n)$ for a balanced tree. The search time grows slowly because each comparison selects a smaller subtree.
- Hash-Based Search: Its expected theoretical complexity is $O(1)$. The search time generally remains nearly constant as the dataset grows, assuming a good hash function and limited collisions.

2. Which Algorithm Scales Better?

Hash-Based Search generally scales best for repeated lookups, followed by balanced BST Search. Brute Force Search scales least efficiently because its search time increases linearly with the number of records.

3. Differences Between Theory and Experiment

The measured execution times may differ from theoretical predictions. For example, hash search time may increase slightly as the dataset grows, even though its expected search complexity is constant.

4. Reasons for the Differences

- Hardware: Processor speed and available memory affect execution time.
- Programming language: Python introduces execution overhead.
- Data structures: Hash collisions and BST structure affect search performance.
- Measurement conditions: Background processes and timing overhead can affect measurements.
- Preprocessing: Building the BST and hash table takes additional time, although the experiment measures search time separately.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

1. Recommended Algorithm

I recommend Hash-Based Search for the university student-record system because it provides fast retrieval of student records using registration numbers.

2. Why It Is Better Than the Alternatives

- **Brute Force Search**: Checks records one by one, making it slower for large datasets.
- **BST Search**: Provides efficient searching, but a balanced BST requires approximately logarithmic search time.
- **Hash-Based Search:** Provides expected constant-time lookup, making it suitable for frequent registration-number searches.

3. Theoretical Efficiency

Hash-Based Search has expected (O(1)) search complexity with a good hash function and controlled collisions. Its worst-case complexity is (O(n)) if many registration numbers collide.

4. Experimental Performance

The execution-time measurements and graphs from Task 9 should be used to compare the algorithms. If Hash-Based Search shows lower search times across the tested dataset sizes, this supports the recommendation.

5. Preprocessing Cost

The hash table must be created and populated before searching. Building it takes expected (O(n)) time and (O(n)) additional memory. Updates are required when student records are added or removed.

6. Trade-offs and Limitations

Hash-Based Search requires additional memory and effective collision handling. It does not naturally maintain records in sorted order, so a balanced BST may be preferable when ordered retrieval is required.

7. Scalability

Hash-Based Search remains practical for databases containing 1,000 to 1,000,000 or more records, provided sufficient memory is available and collisions are managed effectively.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

The original system was designed to search student records using registration numbers. The new requirement is to handle 50,000 searches per hour on a database containing 1,000,000 records, with new records added only once per day.

- **Changed requirement:** The system must handle frequent searches on a large database.
- **Affected assumptions:** Fast searching and efficient handling of repeated requests are now more important.
- **Search frequency:** 50,000 searches per hour is approximately 14 searches per second, so the system needs efficient lookup operations.
- **Record updates:** Since new records are added only once per day, the search structure can be built once and reused for many searches.

B. Evaluation of the Original Design (2 Marks)

The original design uses Hash-Based Search.

- **Search performance:** It provides expected (O(1)) lookup time, making it suitable for frequent registration-number searches.

- **Preprocessing cost:** Building the hash table takes expected (O(n)) time. This cost is acceptable because the table can be reused for many searches.
- **Alternative:** A balanced BST provides (O(log n)) search time, but hashing is generally more efficient for exact registration-number lookups.

Therefore, Hash-Based Search remains suitable for the new requirements.

C. Adaptation or Redesign (2 Marks)

Decision: Modify the original design.

Use a prebuilt hash table with incremental updates.

1. Load all one million student records into the hash table.
2. Use registration numbers as keys to retrieve student records.
3. Reuse the hash table for every search request.
4. Insert new records into the existing table once per day without rebuilding the entire table.

Computational complexity:

- **Initial construction:** Expected (O(n)).
- **Search:** Expected (O(1)).
- **Insertion:** Expected (O(1)) per record.
- **Storage:** (O(n)).

D. Evidence-Based Defence (2 Marks)

- **Mathematical analysis**: Hash-Based Search has expected (O(1)) search time, compared with (O(n)) for Brute Force Search and (O(log n)) for balanced BST Search.
- **Search frequency:** The system must handle 50,000 searches per hour, making fast lookups important.
- **Preprocessing cost:** Building the hash table takes expected (O(n)) time, but it can be reused for many searches. New records are added only once per day.

- **Dataset size:** Hash-Based Search can handle one million records if sufficient memory is available.
- **Experimental evidence:** Compare the actual execution times from Task 9. If Hash-Based Search is faster in your measurements, this supports the recommendation. Otherwise, discuss the observed results and explain the choice using both measured performance and theoretical complexity.